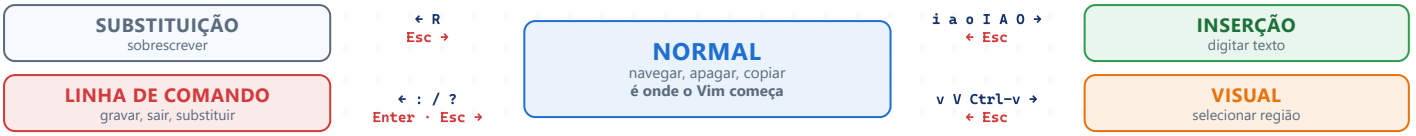
EDITORES NO TERMINAL: VIM E NANO

*Três da manhã, servidor remoto por SSH, o `sshd` não sobe. Sem interface gráfica. Sem `nano` — a imagem mínima não o inclui. Há `vi`, porque **todo** Unix tem `vi`. A pessoa digita a correção e as letras não aparecem.*
Editar no terminal não é preferência: é competência de recuperação.

096
GUIAS LINUX

O VIM É MODAL — E NORMAL É O ESTADO DE REPOUSO



Toda volta é `Esc` — apertada duas vezes, você está em NORMAL a partir de qualquer lugar; é a tecla mais útil do editor. **A regra que resolve o pânico do primeiro contato: se as letras que você digita não aparecem, você está em NORMAL — aperte `i` e digite.** Para sair sem gravar nada: `Esc`, depois `:q!` e Enter. O Vim assume que você passa mais tempo *navegando e transformando* texto do que inserindo caracteres — o que é verdade quando se edita configuração. **Modo de inserção é para escrever; sai-se dele assim que terminou.**

OS DEZ QUE BASTAM — E A GRAMÁTICA QUE DÁ VELOCIDADE

`i`	entra em **inserção** (antes do cursor)	`Esc`	volta para o modo **normal**
`:w`	grava	`:q`	sai
`:wq`	grava e sai (ou **ZZ** em normal)	`:q!`	sai **descartando** as alterações
`dd`	apaga a linha inteira	`u`	desfaz (**Ctrl-r** refaz)
`/txt`	busca; **n** vai à próxima ocorrência	`:123`	vai para a linha 123

Aprendendo só isto você já não fica preso e já corrige uma configuração remota. Depois vem a composição: um **número** na frente multiplica (`3dd`), repetir o operador age na linha (`dd`, `yy`, `cc`), e o `.` **repete a última edição** — o comando mais subestimado do Vim. Busca e substituição têm a sintaxe do `sed`: `:%s/velho/novo/gc` confirma cada troca, `:5,20s/…/g` limita às linhas, `:g/^#/d` apaga os comentários e `:v/erro/d` apaga tudo que **não** casa. E `vim -u NONE` abre sem `.vimrc` nem plugin — a primeira coisa a tentar quando o editor age estranho.

OPERADOR + MOVIMENTO	O QUE FAZ
`dw`	**d** apagar + **w** palavra: apaga até o fim da palavra
`c$`	**c** alterar + **$** fim da linha; `y}` copia o parágrafo
`di"`	apaga **dentro** das aspas, de qualquer ponto entre elas
`cip`	substitui o parágrafo inteiro e já entra em inserção

Os *text objects* (`iw`, `i"`, `i(`, `it`) são o que mais rende em configuração: com o cursor em qualquer ponto entre as aspas, `ci"` apaga o conteúdo e já deixa você digitando. **Um `~/.vimrc` mínimo:** `syntax on`, `number`, `expandtab`, `incsearch` `hlsearch`, `ignorecase` `smartcase` (`erro` acha `ERRO`; `Erro` só acha `Erro`) e `undofile`.

O NANO, E QUEM DECIDE QUAL EDITOR ABRE

NANO	O QUE FAZ (^ = CTRL, M- = ALT)
`Ctrl+O` · `Ctrl+X`	grava (*write Out*) · sai, perguntando se quer gravar
`Ctrl+W` · `Ctrl+\`	busca (`Alt+W` repete) · busca e substitui. `Ctrl+_` vai a uma linha
`Ctrl+K` · `Ctrl+U`	recorta a linha · cola. `Alt+A` seleciona bloco; `Alt+U` / `Alt+E` desfaz e refaz
`~/.nanorc`	`linenumbers`, `constantshow` e — o que importa — `tabstospaces`

O `tabstospaces` evita uma classe inteira de erro: YAML com tabulação real é inválido, e a mensagem raramente diz isso. **E quem decide qual editor abre é a variável `EDITOR`** — `git commit`, `crontab -e`, `visudo` e `systemctl edit` não escolhem editor: abrem o que estiver lá. É por isso que tanta gente se vê presa no Vim sem ter pedido, já que o padrão do Debian e do Kali é o `vi` quando a variável não existe. `export EDITOR=nano` no seu perfil resolve **para você**; `update-alternatives --config editor` muda o `/usr/bin/editor` e resolve **para a máquina**.

Valide antes de gravar, sempre: `visudo` para o `sudoers`, `sshd -t` para o SSH, `nginx -t`, `systemd-analyze verify` para as units. Um `/etc/sudoers` com erro de sintaxe deixa **ninguém** capaz de usar `sudo`, e recuperar exige boot em modo de recuperação. **Editando o `sshd_config` por SSH, mantenha a sessão atual aberta** e teste numa segunda conexão antes de fechá-la.

`sudo vim /etc/algo` executa o editor inteiro como root — com o *seu* `.vimrc` e os *seus* plugins. Esse arquivo é gravável pelo seu usuário, logo por qualquer processo que rode como você: um plugin comprometido vira execução como root na próxima configuração que você editar. Pior: dentro de um Vim rodando como root, `:!bash` abre um **shell de root** — o que transforma uma permissão de `sudo` restrita a "editar um arquivo" em root completo. A resposta é `sudoedit` (ou `sudo -e`): ele copia o arquivo para um temporário *seu*, roda o editor com os *seus* privilégios e a sua configuração, e só a cópia de volta acontece como root — o editor nunca executa privilegiado. No `sudoers`, conceder `sudoedit /etc/nginx/nginx.conf` é permissão de editar aquele arquivo; conceder `sudo vim` nele é, na prática, **conceder root**.

RESUMO RÁPIDO

- ✓ **Editar no terminal é competência de recuperação.** Todo Unix tem `vi`; imagem mínima e contêiner muitas vezes não têm `nano` — e `apt install` não é opção quando foi a rede que você quebrou.
- ✓ **O Vim é modal, e NORMAL é o repouso.** Letras não aparecem? Aperte `i`. `Esc` volta de qualquer lugar; `:q!` sai descartando tudo.
- ✓ **Dez comandos bastam para sobreviver**; a gramática **operador + movimento** e o `.` de repetição é que dão velocidade depois.
- ✓ **O `nano` mostra os atalhos na tela** e resolve o dia a dia — e `set tabstospaces` evita o YAML inválido por tabulação.
- ✓ **`sudoedit`, nunca `sudo vim`.** O segundo roda o editor inteiro como root, e `:!bash` lá dentro é um shell de root.
- ✓ **`EDITOR` decide o que `git commit`, `crontab -e` e `visudo` abrem** — defina-a no perfil, ou mude o padrão da máquina com `update-alternatives`.

NO KALI

O Kali traz `vim`, `nano`, `vi` e `mousepad` — a imagem mínima e os contêineres, nem sempre: a `kali-linux-core` e as imagens de contêiner podem vir só com o `vim.tiny` (o `vi`), sem `nano`. É exatamente a situação da abertura, e é o argumento prático para saber os dez comandos de sobrevivência mesmo preferindo o `nano`. **O `sudoedit` importa mais aqui do que em outros sistemas:** numa distribuição onde se instalam ferramentas de terceiros o tempo todo, e onde plugins de editor entram junto com repositórios clonados, a superfície do `sudo vim` deixa de ser hipótese. Adote o `sudoedit` como reflexo para tudo em `/etc`: o custo é zero e fecha um caminho de escalada conhecido. **Arquivo de troca (`.swp`) órfão denuncia sessão interrompida — e às vezes atividade que não foi sua.** Morto sem fechar, o Vim deixa um `.arquivo.swp`, e o `E325: ATTENTION` da abertura seguinte oferece recuperar o conteúdo não salvo. Numa investigação de máquina possivelmente comprometida, um `.swp` em `/etc` indica que *alguém editou algo ali* e a sessão foi interrompida: olhe antes de apagar — `find /etc -name '.*.swp'` encontra, `vim -r` recupera. **Fora do Debian:** o `vi` do RHEL mínimo é o `vim-minimal`, ainda mais limitado que o `vim.tiny`, e o `nano` não vem em instalação mínima; lá não existe o alias `editor`, então `EDITOR` é o caminho principal. O `sudoedit` e o `visudo` são idênticos nas duas famílias — vêm do mesmo pacote `sudo` —, e o argumento de segurança vale em qualquer distribuição.